

Submitted by:

Under the Guidance of

Department of Computer Science and Engineering (AI & ML)

JSS Academy of Technical Education

Noida, Uttar Pradesh

Academic Year: 2024-25

RE-DACT: An AI-Driven Platform for Automated Redaction of Sensitive Information in Documents

**Submitted to the Department of Computer Science and
Engineering-AIML**

**in Partial Fulfillment of the Requirements
for the Degree of**

Bachelor of Technology

in

Computer Science and Engineering-AIML

by

Name (Roll No.)

Name (Roll No.)

Name (Roll No.)

Name (Roll No.)

Under the Supervision of

Supervisor Name

Department of Computer Science and Engineering-AIML

JSS Academy of Technical Education ,Noida



JSS ACADEMY OF TECHNICAL EDUCATION,NOIDA

DR. APJ ABDUL KALAM TECHNICAL UNIVERSITY

UTTAR PRADESH, LUCKNOW

(Formerly Uttar Pradesh Technical University, Lucknow)

MAY, 2026

VISION AND MISSION

VISION OF THE INSTITUTE

JSS Academy of Technical Education, Noida aims to become an institution of excellence in outcome-based education by helping students build knowledge, skills, research aptitude, and ethical values for solving contemporary engineering problems.

MISSION OF THE INSTITUTE

1. Develop a platform for achieving globally acceptable levels of intellectual acumen and technological competence.
2. Create an inspiring academic environment that supports quality research.
3. Provide an environment that encourages ethical values and a positive attitude.

VISION OF THE DEPARTMENT

“To spark the imagination of Computer Science engineers with values, skills, and creativity to solve real-world problems.”

MISSION OF THE DEPARTMENT

1. Inculcate creative thinking and problem-solving skills through effective teaching, learning, and research.
2. Empower professionals with core competency in Computer Science and Engineering.
3. Foster independent and lifelong learning with ethical and social responsibility.

PROGRAM OUTCOMES (POs)

Engineering graduates will be able to:

PO1: Engineering knowledge: Apply knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to solve complex engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems to reach substantiated conclusions using mathematics, natural sciences, and engineering sciences.

PO3: Design/development of solutions: Design solutions for complex engineering problems and develop system components or processes that meet specified needs with consideration for public health, safety, cultural, societal, and environmental concerns.

PO4: Conduct investigations of complex problems: Use research-based knowledge and research methods, including design of experiments, analysis and interpretation of data, and synthesis of information, to provide valid conclusions.

PO5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modeling, to complex engineering activities with an understanding of their limitations.

PO6: The engineer and society: Apply contextual knowledge to assess societal, health, safety, legal, and cultural issues and the responsibilities relevant to professional engineering practice.

PO7: Environment and sustainability: Understand the impact of professional engineering solutions in societal and environmental contexts and demonstrate the need for sustainable development.

PO8: Ethics: Apply ethical principles and commit to professional ethics, responsibilities, and norms of engineering practice.

PO9: Individual and teamwork: Function effectively as an individual, and as a member or leader in diverse teams and multidisciplinary settings.

PO10: Communication: Communicate effectively on complex engineering activities with the engineering community and society at large, including writing reports, preparing design documentation, making presentations, and giving and receiving clear instructions.

PO11: Project management and finance: Demonstrate knowledge of engineering and management principles and apply them to one's own work, as a member or leader in a team, to manage projects in multidisciplinary environments.

PO12: Life-long learning: Recognize the need for independent and lifelong learning and develop the ability to pursue it in the broad context of technological change.

PROGRAM EDUCATIONAL OUTCOMES (PEOs)

PEO1: Apply computational skills needed to analyze, formulate, and solve engineering problems.

PEO2: Establish entrepreneurial ability and work in interdisciplinary research and development organizations, individually or as part of a team.

PEO3: Inculcate ethical values and leadership qualities needed for a successful career.

PEO4: Develop analytical thinking for understanding and solving real-world problems and sustain an attitude of lifelong learning for higher education.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: Acquire in-depth knowledge of theoretical foundations and issues in Computer Science to build learning ability and computational skills.

PSO2: Analyze, design, develop, test, and manage complex software systems and applications using advanced tools and techniques.

Course Outcomes (COs)

C410.1: Identify, formulate, design, and analyze a research-based or web-based problem.

C410.2: Communicate effectively in verbal and written form.

C410.3: Apply appropriate computing and engineering skills to solve the formulated problem within the stipulated time.

C410.4: Work effectively as part of a team in multidisciplinary areas.

C410.5: Consolidate the final outcome in the form of a publication.

CO-PO-PSO Mapping

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
C410.1	3	3	3	3	2	3	3	3	3	3	2	3	3	3
C410.2	2	2	2	2	2	2	2	2	2	3	2	3	2	3
C410.3	3	3	3	3	3	3	2	3	3	3	3	3	3	3
C410.4	3	3	3	3	2	3	2	3	3	3	3	3	3	3
C410.5	3	3	3	3	3	3	3	3	3	3	3	3	3	3
C410	2.80	2.80	2.80	2.80	2.40	2.80	2.40	2.80	2.80	3.00	2.60	3.00	2.80	3.00

DECLARATION

We declare that this project report is our own work. To the best of our knowledge, it does not contain material previously published or written by another person, or material submitted for the award of any other degree or diploma, except where proper acknowledgement has been made in the text.

Name : Student Name1

Roll. No. :

(Candidate Signature)

Name : Student Name2

Roll. No. :

(Candidate Signature)

Name : Student Name3

Roll. No. :

(Candidate Signature)

Name : Student Name4

Roll. No. :

(Candidate Signature)

CERTIFICATE

This is to certify that the project report entitled “RE-DACT: An AI-Driven Platform for Automated Redaction of Sensitive Information in Documents” is submitted by the students listed in this report in partial fulfillment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering (Artificial Intelligence and Machine Learning) at Dr. A.P.J. Abdul Kalam Technical University, Uttar Pradesh, Lucknow. The work was carried out under my supervision. To the best of my knowledge, the work presented in this report is original and has not been submitted for the award of any other degree.

Signature

Name of Supervisor

Designation

Address

Date:

ACKNOWLEDGEMENT

We are grateful to **Supervisor Name**, Department of Computer Science & Engineering, JSS Academy of Technical Education, Noida, for guiding this project and reviewing our work at each stage. Their feedback helped us keep the project focused and technically grounded.

We also thank **Dr. Kakoli Banerjee**, Head, Department of Computer Science & Engineering, JSS Academy of Technical Education, Noida, for the support provided by the department during the development of this project.

We thank the faculty members of the department for their suggestions and assistance. We are also thankful to our classmates and friends for their help during testing, documentation, and review.

Name : Student Name1

Roll. No. :

(Candidate Signature)

Name : Student Name3

Roll. No. :

(Candidate Signature)

Name : Student Name2

Roll. No. :

(Candidate Signature)

Name : Student Name4

Roll. No. :

(Candidate Signature)

ABSTRACT

Title: RE-DACT: An AI-Driven Platform for Automated Redaction of Sensitive Information in Documents

Digital documents often carry personally identifiable information (PII) that should not be visible when files are shared, archived, or sent for review. Manual redaction is slow and easy to get wrong, especially when the same team has to process scanned forms, identity cards, and PDFs in large numbers. This project presents RE-DACT, a web-based redaction platform for detecting and masking sensitive Indian identity information such as Aadhaar numbers, PAN numbers, payment card numbers, email addresses, and phone numbers in PDF and image files.

RE-DACT combines OCR, rule-based detection, and validation checks. Tesseract extracts text and character bounding boxes from the input document. The system then validates Aadhaar candidates with the Verhoeff algorithm and payment card candidates with the Luhn algorithm. PAN detection uses the official ten-character structure and checks the fourth character, which represents the PAN holder type. These checks reduce the false positives that would occur with plain regular-expression matching.

The processing pipeline also handles common scanning problems. It tries four orientations and two preprocessing states, giving eight OCR attempts in the worst case. If the first readable orientation produces a detection, the pipeline stops early, which keeps normal cases fast while still allowing rotated documents to be processed. In the project tests, single-page Aadhaar images with clear text were processed in 274–296 ms, while difficult cases that needed more OCR attempts took longer.

The platform provides three masking levels. Standard masking hides all but the last four characters. Partial masking hides the first four characters only, matching the current implementation. Full masking hides the entire detected value. For email addresses, phone numbers, names, and locations detected through optional NER, the system uses full masking because partial exposure is less useful and may still reveal identity.

The implementation uses FastAPI for the REST API, plain HTML/CSS/JavaScript for the browser interface, OpenCV for image operations, pdf2image for PDF conversion, and img2pdf for output assembly. It also records processing events in a JSON-lines audit log. Each entry includes a SHA-256 hash linked to the previous entry for tamper evidence during a running process. Processed files are kept only in temporary working directories, and downloadable results expire after ten minutes.

Testing on a small set of Indian identity-document samples showed reliable Aadhaar detection when the text was clear and readable. PAN, payment card, and NER detections depended more heavily on OCR quality, especially on images with complex backgrounds or stylized text. The report documents the system design, validation algorithms, implementation details, test results, limitations, and future improvements such as stronger NER, better multilingual OCR, batch processing, and containerized deployment.

Keywords: Redaction, PII, OCR, Verhoeff Algorithm, Luhn Algorithm, Tesseract, Privacy, Audit Logging, FastAPI

LIST OF FIGURES

LIST OF TABLES

2.1	Literature Gaps and RE-DACT's Response	10
4.1	Software Requirements Summary	22
4.2	Detection Results Summary	24
4.3	Processing Time Results	25
4.4	Test Case Execution Summary	27
4.5	Qualitative Comparison with Related Redaction Approaches	28

List of Symbols and Abbreviations

TABLE OF CONTENTS

ABSTRACT	x
Declaration	xvii
Certificate	xvii
Acknowledgement	xvii
Abstract	xvii
List of Tables	xvii
List of Figures	xvii
List of Symbols and Abbreviations	xvii
 CHAPTER 1 INTRODUCTION	 1
1.1 Problem Statement	1
1.2 Motivation	2
1.3 Project Objectives	3
1.4 Scope of Project	4
1.5 Organization of the Report	5
 CHAPTER 2 LITERATURE REVIEW	 6
2.1 Background and Terminology	6
2.2 Traditional Redaction Approaches	7
2.3 AI-Based Approaches for PII Detection	7
2.4 Related Work in Document Redaction	8
2.5 Algorithms for PII Validation	9
2.6 Research Gap Analysis	10
2.7 Summary	10
 CHAPTER 3 SYSTEM DESIGN AND METHODOLOGY	 12
3.1 System Architecture Overview	12

3.2	PII Detection Module Design	13
3.3	Verhoeff Algorithm for Aadhaar Validation	14
3.4	Luhn Algorithm for Payment Card Validation	14
3.5	PAN Card Detection with Entity Validation	15
3.6	Named Entity Recognition for Unstructured PII	16
3.7	Multi-Orientation OCR Pipeline	16
3.8	Masking Strategies and Implementation	18
3.9	Overlap Deduplication Logic	18
3.10	Tamper-Evident Audit Logging	19
3.11	System Interfaces and API Design	19
3.12	Summary	20
CHAPTER 4	IMPLEMENTATION AND RESULTS	21
4.1	Implementation Details	21
4.1.1	Software and Hardware Requirements	22
4.1.2	Assumptions and Dependencies	22
4.1.3	Current Constraints	23
4.2	Results	24
4.2.1	Interface Snapshots and Result Views	25
4.2.2	Test Cases	26
4.2.3	Performance Evaluation	26
4.3	Summary	28
CHAPTER 5	CONCLUSION AND FUTURE SCOPE	29
5.1	Conclusion	29
5.2	Future Scope	30
	REFERENCES	32
CHAPTER A	LIST OF PAPERS	34
A.1	List of Published Papers	34

A.2	List of Accepted Papers	34
A.3	List of Communicated Papers	34
CHAPTER B PLAGIARISM REPORT		35
B.1	Plagiarism Report	35
B.2	AI Content Report	35

CHAPTER 1

INTRODUCTION

1.1 Problem Statement

Organizations now handle a large number of documents that contain personal data: identity cards, application forms, bank records, medical paperwork, scanned PDFs, and photographs of documents captured on phones. These files often include Aadhaar numbers, PAN numbers, payment card numbers, email addresses, phone numbers, names, and addresses. When such documents are shared without proper redaction, the exposed data can be used for identity theft, fraud, or unauthorized profiling.

Manual redaction is still common, but it does not scale well. A reviewer has to inspect every page, identify sensitive fields, and apply masking without missing a digit or masking the wrong information. The work is repetitive, and mistakes are hard to notice after the document has already been shared. The risk is higher for scanned documents because the sensitive value may appear at an unusual angle, inside a noisy image, or in a layout that differs from the expected template.

The regulatory pressure around privacy has also increased. The GDPR has led to large penalties in Europe, including the EUR 1.2 billion penalty issued against Meta Ireland in May 2023 for unlawful transfers of EU/EEA user data. In India, the Digital Personal Data Protection Act, 2023 sets duties for data fiduciaries and provides monetary penalties that may extend up to Rs. 250 crore for certain breaches. These rules make reliable document handling a practical compliance need, not just a technical preference.

This project focuses on automated redaction for Indian identity-document workflows. Aadhaar is a 12-digit identifier issued to Indian residents. PAN is a ten-character alphanumeric identifier issued by the Income Tax Department, with a fixed structure

and a fourth character that encodes holder status. Payment card numbers follow internationally used numbering structures and can be checked with the Luhn algorithm. These identifiers appear frequently in forms and scanned records, so they are good targets for a validation-based redaction system.

Many existing tools support general redaction, but they usually need manual selection, cloud processing, or custom recognizers for Indian identifiers. Generic pattern matching can also over-redact ordinary numbers because it cannot tell whether a 12-digit sequence is a valid Aadhaar number. RE-DACT addresses this gap by combining OCR with validation checks, orientation handling, configurable masking, and an audit log.

1.2 Motivation

The motivation for RE-DACT comes from a simple problem: sensitive Indian identity data is easy to collect and hard to handle safely once it appears inside documents. Government offices, educational institutions, banks, clinics, and private businesses may all receive identity proofs during onboarding, verification, claims, admissions, or compliance checks. A small error during sharing can expose data that should have been hidden.

The project also shows how established tools can be combined into a practical system. Tesseract provides OCR and character-level bounding boxes. OpenCV supports image rotation and masking. Verhoeff and Luhn validation add deterministic checks for structured numbers. FastAPI makes the processing pipeline available through a browser interface and REST endpoints. None of these pieces is new by itself; the value is in connecting them in a way that works for the target document types.

Document orientation is another reason for building the system. Real inputs are rarely perfect. A user may upload a phone photograph, a sideways scan, or a PDF made from images with inconsistent page rotation. A redaction tool that assumes upright text can miss sensitive values. RE-DACT therefore tries multiple orientations and uses an early-exit strategy so normal inputs do not pay the cost of all OCR attempts.

The project is also useful as a learning exercise in applied AI and privacy engineering. It brings together OCR, deterministic validation, web APIs, frontend design,

temporary file handling, and hash-based audit logging. The result is small enough to understand, but complete enough to show the trade-offs involved in real document processing.

1.3 Project Objectives

The main objective of this project is to design and implement a web-based platform that detects and masks sensitive information in PDF and image documents, with a focus on Indian identity data.

The specific objectives are:

1. Build a PII detection module for Aadhaar, PAN, payment card numbers, email addresses, phone numbers, and optional NER-based names and locations.
2. Validate Aadhaar candidates with the Verhoeff checksum and payment card candidates with the Luhn checksum to reduce false positives.
3. Detect PAN numbers through structural pattern matching and fourth-character holder-type validation.
4. Process PDF, JPG, JPEG, and PNG files through a pipeline that preserves page structure where possible.
5. Handle rotated inputs by trying four orientations and two preprocessing states.
6. Provide three masking levels: standard, partial, and full.
7. Expose the system through REST API endpoints and a browser interface.
8. Record processing events in a tamper-evident audit log using SHA-256 hash chaining.
9. Measure detection behavior and processing time on representative sample documents.

1.4 Scope of Project

The project includes OCR-based redaction for structured Indian identifiers and a limited set of general PII types. The implemented system processes PDF, JPG, JPEG, and PNG files. PDF pages are converted to images at 300 DPI, processed page by page, and assembled back into a PDF when redaction is applied. Image inputs are processed directly.

The detection scope includes Aadhaar numbers, payment card numbers, PAN numbers, email addresses, and Indian phone-number-like patterns. When the optional transformer dependencies are installed, the system can also attempt NER-based detection of names and locations through the `dslim/bert-base-NER` model. This NER support is useful but should be treated as probabilistic and dependent on OCR quality.

The masking scope includes three levels. Standard masking hides all but the last four characters. Partial masking hides the first four characters only, which is the behavior implemented in the current code and user interface. Full masking hides the entire detected value. For names, locations, email addresses, and phone numbers, the implementation masks the full value regardless of the selected level.

The system stores uploaded and processed files only in temporary working directories. Redacted results remain available for download for ten minutes and are then cleaned up. This is a short-retention design rather than permanent storage. The audit log stores metadata such as filename, file type, detection counts, processing time, and hash-chain fields; it does not store the document contents.

The project does not currently support video redaction, handwritten text recognition, Word documents, multi-page TIFF files, scheduled batch jobs, or distributed processing. It also does not verify whether a PAN or Aadhaar number is actually issued to a person. The checks confirm format and checksum validity where applicable, not identity authenticity.

The implementation assumes that the input contains reasonably clear printed or typed text. Low-resolution images, glare, stylized fonts, strong background graphics, and poor contrast can reduce OCR quality and therefore reduce detection accuracy. The system is suitable for an academic prototype and moderate local use; high-volume production deployment would need explicit file-size limits, rate limiting, persistent job

storage, stronger audit-chain recovery across restarts, and more robust operational monitoring.

1.5 Organization of the Report

This report is organized into five chapters.

Chapter 1 introduces the problem, motivation, objectives, and scope of RE-DACT.

Chapter 2 reviews related work in document redaction, OCR, PII detection, validation algorithms, and privacy requirements.

Chapter 3 explains the system design and methodology, including the OCR pipeline, validation logic, masking behavior, overlap handling, audit logging, and API design.

Chapter 4 describes the implementation and presents test results, processing-time observations, interface behavior, and comparison with related tools.

Chapter 5 concludes the report and outlines future improvements such as stronger NER support, multilingual OCR, video redaction, batch processing, and containerized deployment.

The References section lists the sources used for the technical and regulatory background. The appendices contain institutional material such as paper and plagiarism-report sections where required.

CHAPTER 2

LITERATURE REVIEW

2.1 Background and Terminology

Document redaction is the process of removing or obscuring information that should not be disclosed. In digital documents this usually means masking text, deleting hidden metadata, or removing image regions that reveal sensitive data. A successful redaction should protect the sensitive value while keeping the rest of the document useful.

Personally identifiable information (PII) is any information that can identify, contact, locate, or distinguish a person, either by itself or in combination with other information. In Indian document workflows, common PII includes Aadhaar numbers, PAN numbers, payment card numbers, phone numbers, email addresses, names, and addresses.

Aadhaar is a 12-digit identifier issued to Indian residents. PAN is a ten-character alphanumeric identifier issued by the Income Tax Department. The official PAN structure uses five letters, four digits, and a final alphabetic check digit. The fourth character represents the holder type, such as individual, company, firm, trust, government, or Hindu Undivided Family. Payment card numbers usually contain 13–19 digits and can be checked with the Luhn algorithm.

Redaction systems are usually judged by precision and recall. Precision measures how often the system masks something that really should be masked. Recall measures how often the system catches all the sensitive content that is present. In privacy work, missed detections are especially risky because one unmasked identifier can defeat the purpose of the whole redaction.

2.2 Traditional Redaction Approaches

Manual redaction is simple to understand but difficult to perform consistently. A reviewer opens a file, searches for sensitive fields, and masks them one by one. This can work for a small number of documents, but it becomes slow and error-prone when the document volume grows or when scans vary in quality. Manual work also depends on the reviewer knowing every identifier format that may appear in the document.

Regular expressions are the most common first step in automated redaction. They are fast, transparent, and easy to audit. A pattern such as `[A-Z]{5}[0-9]{4}[A-Z]` can find text that looks like a PAN number, while a digit pattern can find Aadhaar or payment-card candidates. Regex-based detection is useful because the logic is visible and easy to change.

The weakness of pure pattern matching is that it cannot tell whether a matching string is valid. A random 12-digit number can look like an Aadhaar number. A 10-character string can look like a PAN number without being meaningful. Formatting differences also complicate matching: users may write numbers with spaces, dashes, or OCR errors. Because of this, pattern matching works best when it is followed by validation and, where needed, contextual checks.

OCR adds another layer of difficulty. Scanned documents and photographs have noise, skew, blur, shadows, or rotation. OCR output may confuse similar characters such as 0 and O, 1 and I, or 5 and S. Any downstream detector depends on the text that OCR produces, so poor OCR quality directly affects redaction accuracy.

2.3 AI-Based Approaches for PII Detection

Modern OCR engines have improved document processing substantially. Tesseract 5.x is an open-source OCR engine that uses LSTM-based recognition and supports different page segmentation modes. RE-DACT uses Tesseract through `pytesseract` because it can return character bounding boxes, which are needed for character-level masking.

Named Entity Recognition (NER) can detect unstructured entities such as people and locations. RE-DACT optionally uses the Hugging Face `dslim/bert-base-NER` model, which is a BERT model trained for NER on the CoNLL-2003 dataset. It

can identify PERSON and LOCATION labels, which the system maps to **name** and **address** categories. This is helpful for names and location-like text, but the model was trained on news text and is not a specialized Indian identity-document model. Its results should therefore be treated as supportive rather than definitive.

Large language models have also been studied for PII redaction. Recent work such as PRvL evaluates LLM architectures for redaction accuracy, semantic preservation, latency, and leakage risk. Other work has combined OCR with LLMs for domain-specific redaction, such as drug-label redaction under Thailand’s PDPA. These approaches show promise, but they can be expensive to run and may raise privacy concerns if sensitive documents are sent to external services.

For structured identifiers, deterministic validation remains attractive. A checksum algorithm is fast, explainable, and does not require training data. This is why RE-DACT uses algorithmic validation for Aadhaar and payment card candidates, while keeping NER optional for unstructured fields.

2.4 Related Work in Document Redaction

Several tools and studies informed the design of RE-DACT.

Adobe Acrobat Pro supports manual PDF redaction and tools for removing hidden information. It is mature and widely used, but it is not designed specifically around Indian identity-number validation.

Microsoft Presidio provides an open-source framework for detecting and anonymizing PII. It includes predefined recognizers for common entities and supports custom recognizers. Presidio also validates credit-card-like numbers with pattern and checksum logic. It is a strong general-purpose base, but Indian Aadhaar and PAN handling require custom work.

Sánchez and Batet’s C-sanitized model examines privacy guarantees for document sanitization using information-theoretic ideas. Lee and Woods discuss automated redaction in digital archives and the need for workflows that fit the data source. Orekondy et al. focus on image redaction through segmentation and show the privacy-utility trade-off in visual content.

Thetbanthad et al. present an OCR-plus-LLM pipeline for redacting PII from

drug labels for Thailand’s Personal Data Protection Act. Their work is useful because it treats redaction as a complete pipeline: text extraction, classification, masking, and evaluation against annotated data. RE-DACT follows the same broad idea of combining OCR with downstream detection, but it uses deterministic validation for structured Indian identifiers instead of relying primarily on an LLM.

Peng et al. compare manual redaction with AI-assisted redaction tools and report that the AI-assisted tool in their experiment completed redaction faster and with higher accuracy than manual methods. This supports the practical direction of automation, while also reminding that redaction tools should be evaluated against real documents rather than only described conceptually.

2.5 Algorithms for PII Validation

The Verhoeff algorithm was developed by Jacobus Verhoeff for decimal error detection. It uses multiplication and permutation tables based on the dihedral group D_5 . For Aadhaar validation, the digits are processed from right to left, and a final checksum value of zero indicates that the number passes the check. The algorithm catches common errors such as single-digit mistakes and adjacent transpositions. In RE-DACT, a 12-digit candidate is reported as Aadhaar only if it passes this check and does not match the additional project-specific 1947 ending filter.

The Luhn algorithm is a modulus-10 checksum used for payment card numbers and other identifiers. It doubles alternating digits from the right, subtracts 9 from doubled values above 9, and checks whether the total is divisible by 10. The algorithm is lightweight and catches many ordinary transcription errors. RE-DACT applies it to digit sequences of 13–19 characters before reporting a payment card detection.

PAN detection is different because PAN numbers do not expose a public checksum validation method in the same way. RE-DACT uses structural validation: five upper-case letters, four digits, and a final uppercase letter. It then checks the fourth character against the set of valid holder-type characters: A, B, C, F, G, H, J, L, P, and T. This does not prove that the PAN exists, but it is stronger than a simple alphanumeric regex.

2.6 Research Gap Analysis

The reviewed systems show that redaction is not a single problem. A useful tool needs OCR, detection logic, masking, file handling, and some kind of auditability. General tools already handle parts of this problem well, but they often need customization for Indian identifiers and rotated scans.

RE-DACT addresses a narrower, practical gap. It combines OCR with Verhoeff validation for Aadhaar, Luhn validation for payment cards, structural PAN checks, multi-orientation OCR, configurable masking, and JSON-lines audit logging. The contribution is not a new checksum or a new OCR engine. It is the integration of these pieces into a small web platform aimed at Indian identity-document redaction.

Table 2.1: Literature Gaps and RE-DACT’s Response

Gap Area	Common Limitation	RE-DACT Response
Indian PII support	General tools often need custom rules for Aadhaar and PAN	Built-in Aadhaar, PAN, and payment card detectors
Validation	Regex alone can flag invalid numbers	Verhoeff and Luhn checks reduce false positives
Rotated scans	OCR can fail when text is sideways or upside down	Eight-attempt OCR pipeline with early exit
Auditability	Basic logs may not reveal later changes	Hash-linked JSON-lines audit entries
Deployment simplicity	Some solutions require commercial software or cloud services	Local FastAPI application with browser UI

2.7 Summary

The literature shows that redaction systems need both recognition and judgment. OCR extracts text, but it may be noisy. Regex finds candidates, but it can over-match.

NER adds context, but it is probabilistic and domain-sensitive. Checksum validation is narrow, but it is reliable for identifiers that support it.

RE-DACT uses these methods where they fit best. It relies on deterministic validation for structured identifiers, optional NER for names and locations, and multi-orientation OCR for practical scan handling. The next chapter explains how these choices are implemented in the system design.

CHAPTER 3

SYSTEM DESIGN AND METHODOLOGY

3.1 System Architecture Overview

RE-DACT is designed as a small web application around a document-processing pipeline. The user uploads a PDF or image, selects a redaction level, optionally enables NER, and receives a redacted output file when the system finds PII.

The system has three main layers. The presentation layer is the browser interface in the `static` directory. It handles file selection, redaction-level selection, NER enablement, progress display, detection summaries, preview, and download. The API layer is implemented in `app.py` using FastAPI. It validates uploaded files, creates a temporary working directory, calls the redaction pipeline, stores downloadable results for a short time, and writes audit entries. The processing layer is split across `redaction.py`, `detectors.py`, `ner_detector.py`, and `audit.py`.

The pipeline has six practical stages:

1. File ingestion and extension validation.
2. PDF-to-image conversion at 300 DPI for PDF inputs.
3. OCR through Tesseract using multiple orientations and preprocessing states.
4. PII detection through checksum validation, structural matching, regex, and optional NER.
5. Character-level masking using OCR bounding boxes.
6. Output assembly and audit logging.

The implementation favors simple, inspectable components. Aadhaar and payment card validation are pure Python functions. Image rotation and masking use OpenCV.

PDF conversion uses `pdf2image`, and PDF assembly uses `img2pdf`. The browser interface uses plain HTML, CSS, and JavaScript rather than a frontend framework.

The application uses temporary directories for uploaded and processed files. When redaction succeeds, the result is stored in an in-memory dictionary and remains downloadable for ten minutes. The cleanup routine removes expired working directories during later upload requests. This is a short-retention design. It avoids permanent document storage, but it does not mean the document exists only in memory.

3.2 PII Detection Module Design

The detection module is implemented in `detectors.py`. Each detector accepts a text string and returns a list of dictionaries with a common structure: `type`, `value`, `start index`, `end index`, `length`, and `confidence`. This shared output format lets the masking code treat detections consistently.

The rule-based detectors cover five PII types:

1. Aadhaar numbers, validated with Verhoeff checksum.
2. Payment card numbers, validated with Luhn checksum.
3. PAN numbers, validated through structure and fourth-character holder type.
4. Email addresses, detected with regex.
5. Indian phone-number-like strings, detected with regex.

The optional NER detector adds two more categories: names and address/location-like text. It uses the Hugging Face `dslim/bert-base-NER` model when `transformers` and `torch` are installed. If these dependencies are missing, the system continues with rule-based detection only.

Detection begins with candidate extraction. For Aadhaar and payment cards, the system searches for continuous digit sequences. For PAN, it searches for the official alphanumeric structure. The current implementation does not normalize spaced or dashed Aadhaar/PAN strings before detection, so such formats are a limitation. Candidate strings then pass through the relevant validation check before being reported.

After all detectors run, the module removes overlapping detections. It sorts detections by start position. If a new detection overlaps the most recently retained detection, the one with higher confidence is kept. If confidence is equal, the longer detection is kept. This rule is simple and predictable, but it also means a longer payment-card detection can replace a shorter overlapping Aadhaar detection when both have the same confidence.

3.3 Verhoeff Algorithm for Aadhaar Validation

The Aadhaar detector uses the Verhoeff algorithm. The algorithm processes digits from right to left and updates a checksum using two lookup tables: a multiplication table and a permutation table. If the final checksum is zero, the number passes validation.

In RE-DACT, `detect_aadhaar` searches for 12-digit candidates, applies `verhoeff_checksum`, and reports only candidates whose checksum is zero. It also rejects values ending in 1947, a project-specific filter included to avoid one observed false-positive pattern.

```

1: Input: OCR text
2: Output: Aadhaar detections
3: Find all 12-digit candidate strings.
4: for each candidate s do
5:    $c \leftarrow VerhoeffChecksum(s)$ 
6:   if  $c = 0$  and s does not end with 1947 then
7:     Add an Aadhaar detection with confidence 1.0.
8:   end if
9: end for
10: return detection list

```

The validation cost is negligible compared with OCR because Aadhaar has a fixed length of 12 digits.

3.4 Luhn Algorithm for Payment Card Validation

The payment card detector uses the Luhn algorithm. It scans digit sequences of 13–19 characters, doubles every second digit from the right, subtracts 9 from doubled digits

above 9, and checks whether the final sum is divisible by 10.

```
1: Input: OCR text
2: Output: payment card detections
3: Find all 13–19 digit candidate strings.
4: for each candidate s do
5:   if LuhnChecksum(s) is true then
6:     Add a payment card detection with confidence 1.0.
7:   end if
8: end for
9: return detection list
```

The Luhn check is not proof that a card is active or issued. It only confirms that the number follows a valid checksum pattern. This is still useful because it rejects many random digit sequences that regex alone would flag.

3.5 PAN Card Detection with Entity Validation

PAN numbers use a fixed ten-character structure. The first five characters are uppercase letters, the next four are digits, and the last character is an uppercase letter. The fourth character represents the holder type. RE-DACT checks the fourth character against the valid set A, B, C, F, G, H, J, L, P, T.

```
1: Input: OCR text
2: Output: PAN detections
3: Find candidates matching five letters, four digits, and one letter.
4: for each candidate s do
5:   if the fourth character is a valid holder-type code then
6:     Add a PAN detection with confidence 1.0.
7:   end if
8: end for
9: return detection list
```

This validation reduces false positives but does not prove that the PAN was issued by the Income Tax Department. It is a format and holder-type check, not an online

verification step.

3.6 Named Entity Recognition for Unstructured PII

The optional NER module handles unstructured PII that cannot be validated with checksums. The implementation uses the `dslim/bert-base-NER` model through the Hugging Face Transformers pipeline. It maps `PER` labels to `name` and `LOC` labels to `address` in the internal output.

The module first checks whether `transformers` and `torch` can be imported. If not, it returns an empty list and the rest of the system continues normally. When the dependencies are present, the FastAPI startup event preloads the model to avoid first-request latency.

The NER function processes long OCR strings in 512-character chunks with a 50-character overlap. This is a practical implementation detail in the current code; it is not token-aware chunking. Each returned entity is filtered by confidence threshold, defaulting to 0.85. Duplicate spans created by overlapping chunks are skipped.

NER improves coverage for names and locations, but it has clear limits. The model was trained on English news text, not Indian identity cards or OCR output. It may miss address fragments, mislabel organizations, or behave poorly on noisy scans. For this reason, the report treats NER as an optional extension rather than the main basis for redaction.

3.7 Multi-Orientation OCR Pipeline

The multi-orientation OCR pipeline is implemented in `Extract_and_Mask_UIDs`. It exists because scanned documents are often rotated. A single OCR pass may fail if the text is sideways or upside down.

For each image, the system converts the input to grayscale and prepares eight variants:

1. original orientation,
2. 90 degrees counterclockwise,

3. 180 degrees,
4. 90 degrees clockwise,
5. blurred original,
6. blurred 90 degrees counterclockwise,
7. blurred 180 degrees,
8. blurred 90 degrees clockwise.

The blur uses a 5×5 Gaussian kernel. Tesseract runs with English OCR, OCR engine mode 3, and page segmentation mode 11 for sparse text. After each OCR attempt, the system reconstructs text from character bounding boxes and runs the PII detectors.

The pipeline stops at the first variant that produces at least one detection. This early exit keeps correctly oriented documents fast. If none of the eight variants produces a detection, the function returns no masked output for that image.

- 1: **Input:** image path and redaction level
- 2: **Output:** masked image path and detections
- 3: Convert image to grayscale.
- 4: Build eight orientation/preprocessing variants.
- 5: **for** each variant **do**
- 6: Extract character boxes with Tesseract.
- 7: Reconstruct text from OCR boxes.
- 8: Run PII detection.
- 9: **if** one or more detections are found **then**
- 10: Mask the selected character boxes on the image.
- 11: Return the masked image path and detections.
- 12: **end if**
- 13: **end for**
- 14: **return** no output and an empty detection list

3.8 Masking Strategies and Implementation

The masking logic is controlled by `get_mask_range`. It returns the range of characters to hide inside each detected value.

For structured identifiers, the current behavior is:

1. Standard: mask all characters except the last four.
2. Partial: mask the first four characters only.
3. Full: mask every character.

For names, locations, email addresses, and phone numbers, the implementation always returns the full range. This prevents partial disclosure of values that do not have a safe or useful fixed reveal pattern.

Masking happens at the character level. Tesseract returns one bounding box per recognized character. For each character selected for masking, OpenCV draws a filled black rectangle over the corresponding box. If OCR was performed on a rotated version of the image, the code rotates the image before masking and rotates it back afterward, so the final output matches the original orientation.

For PDFs, each page is converted to an image and processed separately. Pages with detections are replaced by masked images. Pages without detections are kept as converted page images. If at least one page is redacted, `img2pdf` combines the pages into a new output PDF.

3.9 Overlap Deduplication Logic

Overlap deduplication prevents the system from reporting the same character span multiple times. This can happen when a digit sequence is matched by more than one detector or when NER overlaps with a regex detection.

The algorithm sorts all detections by start index. It then compares each detection with the last retained detection. If the two spans overlap, the higher-confidence detection is retained. If both have the same confidence, the longer span is retained. Adjacent spans are not considered overlapping.

This rule is intentionally simple. It avoids complicated priority tables and gives deterministic output. A future version could add explicit detector priorities, for example preferring Aadhaar over payment card when a valid Aadhaar is embedded in a longer digit sequence.

3.10 Tamper-Evident Audit Logging

The audit module writes one JSON object per line to `audit.log` by default, or to the path set in `AUDIT_LOG_PATH`. Each entry records timestamp, request ID, filename, file type, redaction level, detection counts, pages processed, processing time, action, previous hash, and entry hash.

The hash chain works by storing the previous entry's hash in `prev_hash`. The current entry is serialized with sorted keys before the `hash` field is added, and SHA-256 is computed over that canonical JSON string. The resulting hash is then written into the entry.

This design makes later edits detectable if the log is verified in sequence. However, the current implementation keeps the previous hash only in process memory and initializes it to 64 zero characters when the application starts. That means the chain is continuous during a running process, but a production version should read the last existing log entry during startup if it needs an uninterrupted chain across restarts.

Concurrent writes are protected by a threading lock. The audit log stores metadata and detection counts, not the file contents or raw detected values.

3.11 System Interfaces and API Design

The REST API exposes three endpoints.

`GET /api/health` returns application status, version, and whether NER dependencies are available.

`POST /api/process-file` accepts multipart form data. It requires a file and accepts optional fields for `level`, `use_ner`, and `confidence_threshold`. Supported file extensions are PDF, JPG, JPEG, and PNG. Valid redaction levels are `standard`, `partial`, and `full`. Unsupported file types or invalid levels return HTTP 400 errors.

GET `/api/result/{request_id}` returns the processed file if the result still exists and has not expired. If the result is missing or older than the ten-minute TTL, the endpoint returns HTTP 404.

The browser interface uses these endpoints directly. It shows total detections, pages processed, processing time, selected redaction level, file type, a detection-type breakdown, a detections table, a preview, and a download link when a redacted file is available.

3.12 Summary

The RE-DACT design combines simple pieces: OCR, validation checks, optional NER, character-level masking, temporary result storage, and hash-linked audit entries. The strongest part of the system is structured identifier handling, where Verhoeff and Luhn validation reduce false positives. The more uncertain parts are OCR quality and NER-based unstructured detection, which depend heavily on input quality and model fit.

The next chapter explains how this design is implemented, tested, and evaluated in the working application.

CHAPTER 4

IMPLEMENTATION AND RESULTS

4.1 Implementation Details

The RE-DACT implementation follows the design described in Chapter 3. The codebase is organized into small Python modules and a static browser interface.

The main files are:

1. `app.py`: FastAPI application, file validation, endpoint handling, result TTL storage, and static-file serving.
2. `redaction.py`: OCR, rotation handling, masking, PDF conversion, and output assembly.
3. `detectors.py`: Aadhaar, PAN, payment card, email, phone, overlap filtering, and masking-range logic.
4. `ner_detector.py`: optional Hugging Face NER pipeline for names and locations.
5. `audit.py`: JSON-lines audit logging with SHA-256 hash chaining.
6. `static/`: HTML, CSS, and JavaScript for the user interface.

The implementation keeps the API, detection logic, and image-processing code separate. This makes the detectors easier to inspect and test because they do not depend on the web layer. The code also uses Python type hints in places where structured values are important, such as the `ResultEntry` dataclass used for storing downloadable results.

The FastAPI endpoint itself is asynchronous, but the OCR and image processing are CPU-bound operations. In production, concurrency would depend on the number

of workers and available CPU resources. The current implementation is suitable for local use and moderate academic testing.

4.1.1 Software and Hardware Requirements

The system depends on both Python packages and system tools. Tesseract OCR is required for text recognition. Poppler utilities are required by `pdf2image` for PDF-to-image conversion. The Python dependencies are listed in `requirements.txt`.

Table 4.1: Software Requirements Summary

Component	Requirement	Notes
Python	3.11 recommended	Project code also uses common Python 3.x features
Tesseract OCR	5.x recommended	OCR engine used through <code>pytesseract</code>
Poppler	Installed system utility	Required for PDF conversion
FastAPI	0.100.0+	Web framework listed in requirements
OpenCV	4.10.0+	Headless build used for image operations
pdf2image	1.16.3+	Converts PDF pages to images
img2pdf	0.5.1+	Rebuilds redacted PDF output
transformers, torch	Optional	Needed only for NER support

For normal image processing, a consumer laptop with 8 GB RAM is sufficient. Optional NER support needs more memory because the BERT model and PyTorch runtime must be loaded. Processing time depends mainly on OCR, page count, image quality, and whether the first orientation produces a detection.

4.1.2 Assumptions and Dependencies

The system assumes that documents contain printed or typed text. Handwritten identifiers are outside the current scope. OCR quality also depends on resolution, contrast, blur, glare, font style, and background design.

The default OCR language is English (`-l eng`). This works for alphanumeric identifiers such as Aadhaar, PAN, and payment cards, but it does not provide full support for Indian-language text. Documents containing Devanagari, Tamil, Telugu, Kannada, or other scripts would need extra Tesseract language configuration and validation work.

The upload endpoint accepts PDF, JPG, JPEG, and PNG files. It does not currently enforce an explicit maximum file size. Result files are held in temporary folders and are available for ten minutes through `/api/result/{request_id}`. After expiry, cleanup removes the temporary directory.

The audit log is append-only in normal operation, but the current hash chain is initialized when the process starts. A deployment that needs an unbroken audit chain across restarts should load the last hash from the existing log at startup.

4.1.3 Current Constraints

The implementation has several known constraints:

1. OCR errors can cause missed detections.
2. Spaced or dashed Aadhaar and PAN values are not normalized before detection.
3. PAN validation checks structure and holder-type character only; it does not verify issuance.
4. NER is optional and domain-sensitive.
5. Results are stored in memory for download lookup, so a process restart clears active download links.
6. There is no batch-processing queue, rate limit, user authentication, or persistent job database.
7. The audit chain is process-continuous, not restart-continuous, in the current code.

These constraints are acceptable for a focused academic prototype, but they should be addressed before using the system in a high-volume or regulated production setting.

4.2 Results

The evaluation used a small set of sample documents containing Aadhaar, PAN, payment card, email, phone, name, and address-like content. Each file was processed through the complete pipeline, and the output was manually checked against the expected detections. Because the dataset is small, the results should be read as prototype evidence rather than a broad accuracy claim.

Aadhaar detection performed best on clear images. The Verhoeff checksum rejected invalid 12-digit sequences and reduced false positives. PAN and payment card detection depended more heavily on OCR quality. PAN cards with clean text were detected more reliably than images with complex backgrounds or stylized fonts. Payment card detection worked when the card number was read correctly and passed the Luhn check.

Email and phone detection worked well on simple text because both rely on direct regex patterns. NER detections were more variable. Names were easier to detect than full addresses, which is expected because the selected BERT NER model is trained on general English NER categories, not postal-address extraction from OCR output.

Table 4.2: Detection Results Summary

PII Type	Test Docs	Detected	Recorded Accuracy	Notes
Aadhaar	5	5	100%	Clear readable text; Verhoeff validation used
PAN Card	4	2	50%	OCR struggled with complex card backgrounds
Payment Card	3	2	67%	Depends on clean OCR and Luhn validation
Email	3	3	100%	Regex-based detection
Phone	3	3	100%	Regex-based Indian phone pattern
NER Names	3	2	67%	Optional BERT NER; depends on OCR text
NER Address/Location	3	1	33%	Address format variation affects detection

Processing time reflected the early-exit design. Correctly oriented Aadhaar images

were processed in 274–296 ms in the recorded tests. Harder cases took longer because the pipeline had to try more OCR variants.

Table 4.3: Processing Time Results

Document	Type	Time (ms)	Detections	Notes
Aadhaar-1	PNG	296	1	Early exit after first useful OCR attempt
Aadhaar-2	PNG	274	1	Early exit after first useful OCR attempt
Aadhaar-3	PNG	274	1	Early exit after first useful OCR attempt
PAN-Card-1	JPG	1582	0	OCR attempts completed without detection
Screenshot-Aadhaar	JPG	304	1	Slightly slower but still near target range
Credit-Card-1	PNG	699	0	Multiple OCR attempts required

Masking output matched the implementation. For a 12-character identifier such as 123456789012, standard masking produces XXXXXXXX9012, partial masking produces XXXX56789012, and full masking produces XXXXXXXXXXXX. The same masking ranges are used for drawing black rectangles on the image.

For multi-page PDFs, the system processes each converted page separately and records the page number for each detection. If only one page contains PII, the redaction is applied to that page while the other converted pages are carried into the output PDF.

4.2.1 Interface Snapshots and Result Views

The browser interface supports drag-and-drop upload, file removal, redaction-level selection, an NER toggle, and a process button. After processing, it shows the total number of detections, pages processed, processing time, selected level, and file type.

The results section includes a detection breakdown by type and a table showing

the detected entity type, masked value, and page number. When a redacted file is available, the interface shows a preview and a download button. For PDFs, the preview is displayed in an iframe. For image inputs, the redacted image is shown directly.

When no PII is found, the API returns a successful response with `download_url` set to null and a message saying that no PII was found. The interface treats this as a clean result rather than an error.

Error cases are handled through HTTP status codes and UI messages. Unsupported file types and invalid redaction levels return HTTP 400. Processing failures return HTTP 500 with the error detail generated by the backend.

4.2.2 Test Cases

The test cases cover expected inputs, negative cases, error handling, orientation handling, and timing. The table below summarizes the recorded manual test execution.

The test set is not large enough to claim production-level accuracy. It does show that the implemented pipeline works as intended across the main feature paths.

4.2.3 Performance Evaluation

OCR dominates processing time. The checksum and regex detectors run quickly compared with Tesseract. The early-exit design is therefore important: a normal upright image can complete after the first OCR pass, while a difficult image may require several attempts.

For image files, processing time depends on resolution and OCR quality. For PDFs, time increases with page count because each page is converted and processed separately. The current implementation processes PDF pages sequentially. Parallel page processing is a possible future optimization.

NER adds substantial overhead when enabled. The model is preloaded at startup if dependencies are available, which reduces first-request delay, but inference still takes longer than rule-based detection. Users who only need structured identifier redaction can disable NER through the UI or API.

Audit logging adds little overhead compared with OCR. It performs JSON serialization and SHA-256 hashing for one metadata entry per request.

Table 4.4: Test Case Execution Summary

Test Case	Input	Expected	Actual	Status
TC-01	Clear Aadhaar PNG	One Aadhaar detection	One Aadhaar detection	Pass
TC-02	Single-page PDF	Redaction applied to PDF output	Redaction applied	Pass
TC-03	Multiple PII types	All readable supported types detected	All expected readable types detected	Pass
TC-04	Partial masking	First four characters masked	First four characters masked	Pass
TC-05	Full masking	All characters masked	All characters masked	Pass
TC-06	No-PII document	Zero detections	Zero detections	Pass
TC-07	Invalid Aadhaar	Rejected by Verhoeff validation	Rejected	Pass
TC-08	Names/locations with NER enabled	NER detections where confidence is high	NER detections returned	Pass
TC-09	Word document	HTTP 400 unsupported file type	HTTP 400	Pass
TC-10	Invalid level	HTTP 400 invalid level	HTTP 400	Pass
TC-11	Corrupted image	Processing error response	HTTP 500	Pass
TC-12	90-degree rotated Aadhaar	Detection after orientation handling	Detection returned	Pass
TC-13	180-degree rotated Aadhaar	Detection after orientation handling	Detection returned	Pass
TC-14	Upside-down Aadhaar	Detection after orientation handling	Detection returned	Pass
TC-15	Standard Aadhaar timing	Around or below 300 ms	274–296 ms	Pass
TC-16	Difficult orientation/no-detection case	Under 2000 ms	1582 ms	Pass

Table 4.5: Qualitative Comparison with Related Redaction Approaches

Capability	RE-DACT	Adobe Acrobat Pro	Microsoft Presidio	Manual Redaction
Aadhaar Verhoeff validation	Yes	No built-in Indian-specific flow	Custom recognizer required	Depends on reviewer
PAN structure validation	Yes	No built-in Indian-specific flow	Custom recognizer required	Depends on reviewer
Payment card Luhn validation	Yes	Available through search/manual tools	Yes for credit-card recognizer	Depends on reviewer
Multi-orientation OCR attempts	Yes	Not the focus of the tool	Not the core text analyzer’s role	Reviewer dependent
Tamper-evident audit metadata	Basic hash chain	Product logging depends on setup	Not built in as a hash chain	Manual records only
Local web API	Yes	No	Yes as a framework	No
No permanent document storage by default	Yes, short temporary TTL	Local file handling	Depends on integration	Depends on process

The comparison shows where RE-DACT is useful: local processing, Indian structured identifiers, and OCR-to-mask output. It is not a replacement for mature PDF editing suites or a full PII-detection platform. It is a focused prototype for a specific redaction workflow.

4.3 Summary

The implementation translates the design into a working FastAPI application with a browser interface. The recorded tests show strong Aadhaar performance on clear inputs, more variable PAN/payment-card performance when OCR is challenged, and optional NER behavior that depends on text quality and model fit.

The main engineering strengths are the simple module boundaries, validation-based structured detection, multi-orientation OCR, and short-lived result storage. The main limitations are OCR sensitivity, limited input formats, lack of batch infrastructure, and audit-chain continuity across restarts. The next chapter summarizes the project and outlines future work.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

RE-DACT implements a working prototype for automated redaction of sensitive information in PDF and image documents. The project focuses on Indian identity-document workflows, especially Aadhaar, PAN, and payment card detection, while also supporting email, phone, and optional NER-based name and location detection.

The strongest part of the system is structured validation. Aadhaar candidates are checked with the Verhoeff algorithm, and payment card candidates are checked with the Luhn algorithm. PAN candidates are checked against the official ten-character structure and valid fourth-character holder-type codes. These checks make the detector more reliable than plain pattern matching for the supported identifier types.

The multi-orientation OCR pipeline addresses a practical problem in scanned-document handling. By trying four rotations and two preprocessing states, the system can process documents that are not upright. The early-exit strategy keeps normal cases faster because the pipeline stops as soon as a variant produces a detection.

The masking implementation works at character level using Tesseract bounding boxes. This lets the system apply standard, partial, and full masking without obscuring more text than necessary. The current behavior is: standard masks all but the last four characters, partial masks the first four characters only, and full masks the whole detected value.

The FastAPI backend and plain browser interface make the prototype easy to run and test. The API returns detection statistics, page counts, masked values, processing time, and a temporary download link. The frontend presents the same information through a simple upload-and-results workflow.

The audit log adds basic tamper evidence by linking JSON-lines entries with SHA-256 hashes. This gives the project a foundation for compliance-style record keeping. The current implementation is process-continuous, so a production version should restore the previous hash from disk after restart if it needs one uninterrupted chain.

The evaluation shows that the system works well on clear Aadhaar samples and that OCR quality is the main limiting factor for PAN, payment cards, and NER. The results are useful for validating the prototype, but the test set is too small to claim broad production accuracy. Larger and more varied datasets would be needed before deploying the system in a high-risk environment.

Overall, the project demonstrates that a practical redaction pipeline can be built from OCR, deterministic validation, image masking, web APIs, and short-retention file handling. It solves a focused problem and also makes the remaining limitations visible, which is valuable for future development.

5.2 Future Scope

The first improvement should be better normalization before detection. Aadhaar and payment card numbers may appear with spaces or dashes, and PAN text may be affected by OCR case errors. A normalization layer that preserves coordinate mapping would improve recall without giving up validation.

NER support should also be strengthened. The current model detects general English names and locations, but it is not trained for Indian identity documents or noisy OCR output. Future work could evaluate Indian or multilingual NER models, add address-specific extraction, and use confidence calibration based on document type.

Multilingual OCR is another important extension. Many Indian documents contain Hindi, Tamil, Telugu, Kannada, Bengali, Malayalam, Marathi, Gujarati, Punjabi, or other scripts. Adding Tesseract language packs and script-aware preprocessing would make the system useful beyond English alphanumeric fields.

The audit log can be made stronger. A production version should load the last hash at startup, include a verification command, protect log file permissions, and optionally export signed audit summaries. This would make tamper evidence easier to demonstrate.

Deployment also needs more work. Containerization with Docker would simplify setup. A background task queue such as Celery or RQ could handle larger files and batch jobs. Multiple workers would improve throughput, but shared audit logging and temporary-file cleanup would need careful design.

Security hardening should include file-size limits, rate limiting, authentication, stricter CORS settings, malware scanning where appropriate, and clearer retention controls. Temporary files should be cleaned even after unexpected failures, and production deployments should avoid exposing debug details in error messages.

Performance can be improved through parallel page processing, OCR configuration tuning, and selective orientation attempts based on image metadata. NER can be disabled by default for workflows that only require structured identifiers, keeping the common path faster.

Finally, the system should be evaluated on a larger labelled dataset. The next evaluation should include different scanners, phone cameras, lighting conditions, document templates, rotations, compression levels, and backgrounds. Precision, recall, false positives, false negatives, and processing time should be reported separately for each PII type.

These improvements would move RE-DACT from a focused academic prototype toward a more dependable document-redaction tool for Indian identity-data workflows.

References

- [1] R. Smith, "An Overview of the Tesseract OCR Engine," in *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.
- [2] Tesseract OCR, "Tesseract User Manual," 2026. [Online]. Available: <https://tesseract-ocr.github.io/tessdoc/>
- [3] J. Verhoeff, *Error Detecting Decimal Codes*. Mathematical Centre Tracts, no. 29. Amsterdam: Mathematisch Centrum, 1969.
- [4] H. P. Luhn, "Computer for Verifying Numbers," U.S. Patent 2,950,048, Aug. 23, 1960.
- [5] Income Tax Department, Government of India, "How PAN is formed and how it gets its unique identity?" [Online]. Available: <https://www.incometaxindia.gov.in/w/how-pan-is-formed-and-how-it-gets-its-unique-identity->
- [6] Government of India, "The Digital Personal Data Protection Act, 2023," India Code. [Online]. Available: <https://www.indiacode.nic.in/bitstream/123456789/22037/1/a2023-22.pdf>
- [7] Data Protection Commission, Ireland, "Data Protection Commission announces conclusion of inquiry into Meta Ireland," May 22, 2023. [Online]. Available: <https://www.dataprotection.ie/media/press-releases/Data-Protection-Commission-announces-conclusion-of-inquiry-into-Meta-Ireland>
- [8] Adobe, "Redact sensitive content in Acrobat Pro," 2025. [Online]. Available: <https://helpx.adobe.com/acrobat/using/removing-sensitive-content-pdfs.html>
- [9] Microsoft, "PII entities supported by Presidio." [Online]. Available: https://microsoft.github.io/presidio/supported_entities/
- [10] Hugging Face, "dslim/bert-base-NER model card." [Online]. Available: <https://huggingface.co/dslim/bert-base-NER>
- [11] D. Sánchez and M. Batet, "C-sanitized: a privacy model for document redaction and sanitization," *Journal of the Association for Information Science and Technology*, vol. 67, no. 1, pp. 148–163, 2016.
- [12] C. A. Lee and K. Woods, "Automated Redaction of Private and Personal Data in Collections: Toward Responsible Stewardship of Digital Heritage," in *Proceedings of*

The Memory of the World in the Digital Age: Digitization and Preservation, Vancouver, BC, Canada, 2012, pp. 298–312.

[13] T. Orekondy, M. Fritz, and B. Schiele, "Connecting Pixels to Privacy and Utility: Automatic Redaction of Private Information in Images," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 8466–8475.

[14] S. Peng, M.-J. Huang, M. Wu, and J. Wei, "Transforming Redaction: How AI is Revolutionizing Data Protection," *arXiv preprint arXiv:2409.15308*, 2024.

[15] L. Garza, A. Kotal, A. Piplai, L. Elluri, P. K. Das, and A. Chadha, "PRvL: Quantifying the Capabilities and Risks of Large Language Models for PII Redaction," *arXiv preprint arXiv:2508.05545*, 2025.

[16] P. Thetbanthad, B. Sathanarugsawait, and P. Praneetpolgrang, "Automated Redaction of Personally Identifiable Information on Drug Labels Using Optical Character Recognition and Large Language Models for Compliance with Thailand's Personal Data Protection Act," *Applied Sciences*, vol. 15, no. 9, p. 4923, 2025.

[17] S. Ramírez, "FastAPI: Modern, Fast Web Framework for Building APIs with Python." [Online]. Available: <https://fastapi.tiangolo.com>

[18] OpenCV, "Open Source Computer Vision Library." [Online]. Available: <https://opencv.org>

[19] python-pdf2image, "pdf2image documentation." [Online]. Available: <https://pdf2image.readthedocs.io>

APPENDIX A

LIST OF PAPERS

A.1 List of Published Papers

A.2 List of Accepted Papers

A.3 List of Communicated Papers

APPENDIX B

PLAGIARISM REPORT

B.1 Plagiarism Report

B.2 AI Content Report